

State

A *state* is the value of an identity at a point in time.

Quite a lot is packed into the previous sentence. Let's unpack the word *value* first. A value is an immutable, persistent data structure. When you can program entirely with values, life is easy, as we saw in [Chapter 4, *Functional Programming*, on page 85](#).

The flow of time makes things substantially more difficult. Are the New York Yankees the same now as they were last year? In 1927? The roster of the Yankees is an *identity* whose value changes over time.

Updating an identity does not destroy old values. In fact, updating an identity has no impact on existing values whatsoever. The Yankees could trade every player, or disband in a fit of boredom, without in any way altering our ability to think about any past Yankees we happen to care about.

Clojure's reference model clearly separates identities from values. Almost everything in Clojure is a value. For identities, Clojure provides four reference types:

- Refs manage *coordinated, synchronous* changes to shared state.
- Atoms manage *uncoordinated, synchronous* changes to shared state.
- Agents manage *asynchronous* changes to shared state.
- Vars manage *thread-local* state.

Each of these APIs is discussed in this chapter. At the end of the chapter, we will develop a sample application. The Snake game demonstrates how to divide an application model into immutable and mutable components.

Before we start, we will review the intersection of state with concurrency and parallelism, as well as comment on the difficulty with traditional lock-based approaches.

5.1 Concurrency, Parallelism, and Locking

A concurrent program models more than one thing happening simultaneously. A parallel program takes an operation that could be sequential and chooses to break it into separate pieces that can execute concurrently to speed overall execution.

There are many reasons to write concurrent or parallel programs:

- For decades, performance improvements have come from packing more power into cores. Now, and for the near future, performance improvements will come from using more cores. Our hardware is itself more concurrent than ever, and systems must be concurrent to take advantage of this power.
- Expensive computations may need to execute in parallel on multiple cores (or multiple boxes) in order to complete in a timely manner.
- Tasks that are blocked waiting for a resource should stand down and let other tasks use available processors.
- User interfaces need to remain responsive while performing long-running tasks.
- Operations that are logically independent are easier to implement if the platform can recognize and take advantage of their independence.

Concurrency makes it glaringly obvious that more than one observer (e.g., thread) may be looking at your data. This is a big problem for languages that complete¹ value and identity. Such languages treat a piece of data as a bank ledger with only one line. Each new operation erases history, potentially corrupting the work of every other thread on the system.

While concurrency makes the challenges more obvious, it is a mistake to assume that multiple observers come into play only with concurrency. If your program ever has two variables that refer to the same data, those variables are different observers. If your program allows mutability at all, then you must think carefully about state.

Mutable languages tend to tackle the challenge by locking and defensive copying. Continuing the ledger analogy: the bank hires guards (locks) to supervise the activities of anybody using a ledger, and nobody is allowed to modify a ledger while anybody else is using it.

1. <http://www.infoq.com/presentations/Simple-Made-Easy>

When the performance becomes really bad, the bank may even ask ledger readers to make their own private copies of the ledger so they can get out of the way and let transactions continue. These copies must still be supervised by the guards!

As irritating as this model sounds, it gets worse at the level of implementation detail. Choosing what and where to lock is a difficult task. If you get it wrong, all sorts of bad things can happen. *Race conditions* between threads can corrupt data. *Deadlocks* can stop an entire program from functioning at all. *Java Concurrency in Practice* [Goe06] covers these and other problems, plus their solutions, in detail. It is a terrific book, but it is difficult to read it and not ask yourself, “Is there another way?”

Clojure’s model for state and identity solves these problems. The bulk of program code is functional. The small parts of the codebase that truly benefit from mutability are distinct and must explicitly select one of four reference models. Using these models, you can split your models into two layers:

- A *functional model* that has no mutable state. Most of your code will normally be in this layer, which is easier to read, easier to test, and easier to parallelize.
- *Reference models* for the parts of the application that you find more convenient to deal with using mutable state (despite its disadvantages).

Let’s get started working with state in Clojure, using the most notorious of Clojure’s reference models: software transactional memory.

5.2 Refs and Software Transactional Memory

Most objects in Clojure are immutable. When you really want mutable data, you must be explicit about it, such as by creating a mutable *reference* (ref) to an immutable object. You create a ref with this:

```
(ref initial-state)
```

For example, you could create a reference to the current song in your music playlist:

```
(def current-track (ref "Mars, the Bringer of War"))
-> #'user/current-track
```

The ref wraps and protects access to its internal state. To read the contents of the reference, you can call deref:

```
(deref reference)
```

The `deref` function can be shortened to the `@` reader macro. Try using both `deref` and `@` to dereference `current-track`:

```
(deref current-track)
-> "Mars, the Bringer of War"
```

```
@current-track
-> "Mars, the Bringer of War"
```

Notice how in this example the Clojure model fits the real world. A track is an immutable entity. It doesn't change into another track when you are finished listening to it. But the *current* track is a reference to an entity, and it does change.

ref-set

You can change where a reference points with `ref-set`:

```
(ref-set reference new-value)
```

Call `ref-set` to listen to a different track:

```
(ref-set current-track "Venus, the Bringer of Peace")
-> java.lang.IllegalStateException: No transaction running
```

Oops. Because refs are mutable, you must protect their updates. In many languages, you would use a *lock* for this purpose. In Clojure, you can use a *transaction*. Transactions are wrapped in a `dosync`:

```
(dosync & exprs)
```

Wrap your `ref-set` with a `dosync`, and all is well.

```
(dosync (ref-set current-track "Venus, the Bringer of Peace"))
-> "Venus, the Bringer of Peace"
```

The `current-track` reference now refers to a different track.

Transactional Properties

Like database transactions, STM transactions guarantee some important properties:

- Updates are *atomic*. If you update more than one ref in a transaction, the cumulative effect of all the updates will appear as a single instantaneous event to anyone not inside your transaction.
- Updates are *consistent*. Refs can specify validation functions. If any of these functions fail, the entire transaction will fail.

- Updates are *isolated*. Running transactions cannot see partially completed results from other transactions.

Databases provide the additional guarantee that updates are *durable*. Because Clojure's transactions are in-memory transactions, Clojure does not guarantee that updates are durable. If you want a durable transaction in Clojure, you should use a database.

Together, the four transactional properties are called ACID. Databases provide ACID; Clojure's STM provides ACI.

If you change more than one ref in a single transaction, those changes are all coordinated to “happen at the same time” from the perspective of any code outside the transaction. So, you can make sure that updates to `current-track` and `current-composer` are *coordinated*:

```
(def current-track (ref "Venus, the Bringer of Peace"))
-> #'user/current-track
(def current-composer (ref "Holst"))
-> #'user/current-composer

(dosync
  (ref-set current-track "Credo")
  (ref-set current-composer "Byrd"))
-> "Byrd"
```

Because the updates are in a transaction, no other thread will ever see an updated track with an out-of-date composer, or vice versa.

alter

The `current-track` example is deceptively easy, because updates to the ref are totally independent of any previous state. Let's build a more complex example, where transactions need to update existing information. A simple chat application fits the bill. First, create a message record that has a sender and some text:

```
src/examples/chat.clj
(defrecord Message [sender text])
```

Now, you can create messages by instantiating the record:

```
(user.Message. "Aaron" "Hello")
-> #:user.Message{:sender "Aaron", :text "Hello"}
```

Users of the chat application want to see the most recent message first, so a list is a good data structure. Create a messages reference that points to an initially empty list:

```
(def messages (ref ()))
```

Now you need a function to add a new message to the front of messages. You could simply deref to get the list of messages, cons the new message, and then ref-set the updated list back into messages:

```
; bad idea
(defn naive-add-message [msg]
  (dosync (ref-set messages (cons msg @messages))))
```

But there is a better option. Why not perform the read and update in a single step? Clojure's `alter` will apply an update function to a referenced object within a transaction:

```
(alter ref update-fn & args...)
```

`alter` returns the new value of the ref within the transaction. When a transaction successfully completes, the ref will take on its last in-transaction value. Using `alter` instead of `ref-set` makes the code more readable:

```
(defn add-message [msg]
  (dosync (alter messages conj msg)))
```

Notice that the update function is `conj` (short for “conjoin”), not `cons`. This is because `conj` takes arguments in an order suitable for use with `alter`:

```
(cons item sequence)
(conj sequence item)
```

The `alter` function calls its `update-fn` with the current reference value as its first argument, as `conj` expects. If you plan to write your own update functions, they should follow the same structure as `conj`:

```
(your-func thing-that-gets-updated & optional-other-args)
```

Try adding a few messages to see that the code works as expected:

```
(add-message (user.Message. "user 1" "hello"))
-> (#:user.Message{:sender "user 1", :text "hello"})

(add-message (user.Message. "user 2" "howdy"))
-> (#:user.Message{:sender "user 2", :text "howdy"}
   #:user.Message{:sender "user 1", :text "hello"})
```

`alter` is the workhorse of Clojure's STM and is the primary means of updating refs. But if you know a little about how the STM works, you may be able to optimize your transactions in certain scenarios.

How STM Works: MVCC

Clojure's STM uses a technique called Multiversion Concurrency Control (MVCC), which is also used in several major databases. Here's how MVCC works in Clojure.

Transaction A begins by taking a *point*, which is simply a number that acts as a unique timestamp in the STM world. Transaction A has access to its own *effectively private* copy of any reference it needs, associated with the point. Clojure's persistent data structures ([Persistent Data Structures, on page 86](#)) make it cheap to provide these effectively private copies.

During Transaction A, operations on a ref work against (and return) the transaction's private copy of the ref's data, called the *in-transaction value*.

If at any point the STM detects that another transaction has already set/alterd a ref that Transaction A wants to set/alter, Transaction A will be forced to retry. If you throw an exception out of the `dosync` block, then Transaction A will abort *without* a retry.

If and when Transaction A commits, its heretofore private writes will become visible to the world, associated with a single point in the transaction timeline.

Sometimes the approach implied by `alter` is too cautious. What if you *don't care* that another transaction altered a reference out from under you in the middle of your transaction? If in such a situation you would be willing to commit your changes anyway, you can beat `alter`'s performance with `commute`.

commute

`commute` is a specialized variant of `alter` allowing for more concurrency:

```
(commute ref update-fn & args...)
```

Of course, there is a trade-off. Commutes are so named because they must be *commutative*. That is, updates must be able to occur in any order. This gives the STM system freedom to reorder commutes.

To use `commute`, simply replace `alter` with `commute` in your implementation of `add-message`:

```
(defn add-message-commute [msg]
  (dosync (commute messages conj msg)))
```

`commute` returns the new value of the ref. However, the last in-transaction value you see from a `commute` will *not* always match the end-of-transaction value of a ref, because of reordering. If another transaction sneaks in and alters a ref that you are trying to commute, the STM will not restart your

transaction. Instead, it will simply run your commute function again, out of order. Your transaction will *never even* see the ref value that your commute function finally ran against.

Since Clojure's STM can reorder commutes behind your back, you can use them only when you do not care about ordering. Literally speaking, this is not true for a chat application. The list of messages most certainly has an order, so if two message adds get reversed, the resulting list will not correctly show the order in which the messages arrived.

Practically speaking, chat message updates are *commutative enough*. STM-based reordering of messages will likely happen on time scales of microseconds or less. For users of a chat application, there are already reorderings on much larger time scales due to network and human latency. (Think about times that you have “spoken out of turn” in an online chat because another speaker's message had not reached you yet.) Since these larger reorderings are unfixable, it is reasonable for a chat application to ignore the smaller reorderings that might bubble up from Clojure's STM.

Prefer alter

Many updates are not commutative. For example, consider a counter that returns an increasing sequence of numbers. You might use such a counter to build unique IDs in a system. The counter can be a simple reference to a number:

```
src/examples/concurrency.clj
(def counter (ref 0))
```

You should not use commute to update the counter. commute returns the in-transaction value of the counter at the time of the commute, but reorderings could cause the actual end-of-transaction value to be different. This could lead to more than one caller getting the same counter value. Instead, use alter:

```
(defn next-counter [] (dosync (alter counter inc)))
```

Try calling next-counter a few times to verify that the counter works as expected:

```
(next-counter)
-> 1

(next-counter)
-> 2
```

In general, you should prefer alter over commute. Its semantics are easy to understand and error-proof. commute, on the other hand, requires that you think carefully about transactional semantics. If you use alter when commute

would suffice, the worst thing that might happen is performance degradation. But if you use `commute` when `alter` is required, you will introduce a subtle bug that is difficult to detect with automated tests.

Adding Validation to Refs

Database transactions maintain consistency through various integrity checks. You can do something similar with Clojure's transactional memory, by specifying a validation function when you create a `ref`:

```
(ref initial-state options*)
; options include:
; :validator validate-fn
; :meta metadata-map
```

The options to `ref` include an optional validation function that can throw an exception to prevent a transaction from completing. Note that `options` is *not* a map; it is simply a sequence of key/value pairs spliced into the function call.

Continuing the chat example, add a validation function to the `messages` reference that guarantees that all messages have non-nil values for `:sender` and `:text`:

`src/examples/chat.clj`

```
(def validate-message-list
  (partial every? #(and (:sender %) (:text %))))

(def messages (ref () :validator validate-message-list))
```

This validation acts like a key constraint on a table in a database transaction. If the constraint fails, the entire transaction rolls back. Try adding an ill-formed message such as a simple string:

```
(add-message "not a valid message")
-> java.lang.IllegalStateException: Invalid reference state

@messages
-> ()
```

Messages that match the constraint are no problem:

```
(add-message (user.Message. "Aaron" "Real Message"))
-> (#:user.Message{:sender "Aaron", :text "Real Message"})
```

Refs are great for coordinated access to shared state, but not all tasks require such coordination. For updating a single piece of isolated data, prefer an `atom`.

5.3 Use Atoms for Uncoordinated, Synchronous Updates

Atoms are a lighter-weight mechanism than refs. Where multiple ref updates can be coordinated in a transaction, atoms allow updates of a single value, uncoordinated with anything else.

You create atoms with `atom`, which has a signature very similar to `ref`:

```
(atom initial-state options?)  
; options include:  
;   :validator validate-fn  
;   :meta metadata-map
```

Returning to our music player example, you could store the current-track in an atom instead of a ref:

```
(def current-track (atom "Venus, the Bringer of Peace"))  
-> #'user/current-track
```

You can dereference an atom to get its value, just as you would a ref:

```
(deref current-track)  
-> "Venus, the Bringer of Peace"
```

```
@current-track  
-> "Venus, the Bringer of Peace"
```

Atoms do not participate in transactions and thus do not require a `dosync`. To set the value of an atom, simply call `reset!`

```
(reset! an-atom newval)
```

For example, you can set `current-track` to "Credo":

```
(reset! current-track "Credo")  
-> "Credo"
```

What if you want to coordinate an update of both `current-track` and `current-composer` with an atom? The short answer is "You can't." That is the difference between refs and atoms. If you need coordinated access, use a ref.

The longer answer is "You can...if you are willing to change the way you model the problem." What if you store the track title and composer in a map and then store the whole map in a single atom?

```
(def current-track (atom {:title "Credo" :composer "Byrd"}))  
-> #'user/current-track
```

Now you can update both values in a single `reset!`

```
(reset! current-track {:title "Spem in Alium" :composer "Tallis"})  
-> {:title "Spem in Alium", :composer "Tallis"}
```

Maybe you like to listen to several tracks in a row by the same composer. If so, you want to change the track title but keep the same composer. `swap!` will do the trick:

```
(swap! an-atom f & args)
```

`swap!` updates `an-atom` by calling function `f` on the current value of `an-atom`, plus any additional args.

To change just the track title, use `swap!` with `assoc` to update only the `:title`:

```
(swap! current-track assoc :title "Sancte Deus")
-> {:title "Sancte Deus", :composer "Tallis"}
```

`swap!` returns the new value. Calls to `swap!` might be retried, if other threads are attempting to modify the same atom. So, the function you pass to `swap!` should have no side effects.

Both refs and atoms perform synchronous updates. When the update function returns, the value is already changed. If you do not need this level of control and can tolerate updates happening asynchronously at some later time, prefer an agent.

5.4 Use Agents for Asynchronous Updates

Some applications have tasks that can proceed independently with minimal coordination between tasks. Clojure *agents* support this style of task.

Agents have much in common with refs. Like refs, you create an agent by wrapping some piece of initial state:

```
(agent initial-state)
```

Create a counter agent that wraps an initial count of zero:

```
(def counter (agent 0))
-> #'user/counter
```

Once you have an agent, you can send the agent a function to update its state. `send` queues an update-fn to run later, on a thread in a thread pool:

```
(send agent update-fn & args)
```

Sending to an agent is very much like commuting a ref. Tell the counter to inc:

```
(send counter inc)
-> #<clojure.lang.Agent@23451c74: 0>
```

Notice that the call to `send` does not return the new value of the agent, returning instead the agent itself. That is because `send` *does not know* the new value. After `send` queues the `inc` to run later, it returns immediately.

Although `send` does not know the new value of an agent, the REPL *might* know. Depending on whether the agent thread or the REPL thread runs first, you might see a 1 or a 0 after the colon in the previous output.

You can check the current value of an agent with `deref/@`, just as you would a ref. By the time you get around to checking the counter, the `inc` will almost certainly have completed on the thread pool, raising the value to 1:

```
@counter
-> 1
```

If the race condition between the REPL and the agent thread bothers you, there is a solution. If you want to be sure that the agent has completed the actions *you sent* to it, you can call `await` or `await-for`:

```
(await & agents)
```

```
(await-for timeout-millis & agents)
```

These functions will cause the current thread to block until all actions sent from the current thread or agent have completed. `await-for` will return `nil` if the timeout expires and will return a non-`nil` value otherwise. `await` has no timeout, so be careful: `await` is willing to wait forever.

Validating Agents and Handling Errors

Agents have other points in common with refs. They also can take a validation function:

```
(agent initial-state options*)
; options include:
;   :validator validate-fn
;   :meta metadata-map
```

Re-create the counter with a validator that ensures it is a number:

```
(def counter (agent 0 :validator number?))
-> #'user/counter
```

Try to set the agent to a value that is not a number by passing an update function that ignores the current value and simply returns a string:

```
(send counter (fn [_] "boo"))
-> #<clojure.lang.Agent@4de8ce62: 0>
```

Everything looks fine (so far) because `send` still returns immediately. After the agent tries to update itself on a pooled thread, it will enter an exceptional state. You will discover the error when you try to dereference the agent:

```
@counter
-> java.lang.Exception: Agent has errors
```

To discover the specific error (or errors), call `agent-errors`, which will return a sequence of errors thrown during agent actions:

```
(agent-errors counter)
-> (#<IllegalStateException ...>)
```

Once an agent has errors, all subsequent attempts to query the agent will return an error. You make the agent usable again by calling `clear-agent-errors`:

```
(clear-agent-errors agent)
```

which returns the agent to its pre-error state. Clear the counter's errors, and verify that its state is the same as before the error occurred:

```
(clear-agent-errors counter)
-> nil
```

```
@counter
-> 0
```

Now that you know the basics of agents, let's use them in conjunction with refs and transactions.

Including Agents in Transactions

Transactions should not have side effects, because Clojure may retry a transaction an arbitrary number of times. However, sometimes you *want* a side effect when a transaction succeeds. Agents provide a solution. If you send an action to an agent from within a transaction, that action will be sent exactly once, if and only if the transaction succeeds.

As an example of where this would be useful, consider an agent that writes to a file when a transaction succeeds. You could combine such an agent with the chat example from [commute, on page 119](#), to automatically back up chat messages. First, create a backup-agent that stores the filename to write to:

```
src/examples/concurrency.clj
(def backup-agent (agent "output/messages-backup.clj"))
```

Then, create a modified version of `add-message`. The new function `add-message-with-backup` should do two additional things:

- Grab the return value of `commute`, which is the current database of messages, in a `let` binding.
- While still inside a transaction, send an action to the backup agent that writes the message database to `filename`. For simplicity, have the action function return `filename` so that the agent will use the same filename for the next backup.

```
(defn add-message-with-backup [msg]
  (dosync
    (let [snapshot (commute messages conj msg)]
      (send-off backup-agent (fn [filename]
                              (spit filename snapshot)
                              filename))
      snapshot)))
```

The new function has one other critical difference: it calls `send-off` instead of `send` to communicate with the agent. `send-off` is a variant of `send` for actions that expect to block, as a file write might do. `send-off` actions get their own expandable thread pool. Never send a blocking function, or you may unnecessarily prevent other agents from making progress.

Try adding some messages using `add-message-with-backup`:

```
(add-message-with-backup (user.Message. "John" "Message One"))
-> (#:user.Message{:sender "John", :text "Message One"})

(add-message-with-backup (user.Message. "Jane" "Message Two"))
-> (#:user.Message{:sender "Jane", :text "Message Two"}
   #:user.Message{:sender "John", :text "Message One"})
```

You can check both the in-memory messages as well as the backup file messages-backup to verify that they contain the same structure.

You could enhance the backup strategy in this example in various ways. You could provide the option to back up less often than on every update or back up only information that has changed since the last backup.

Since Clojure's STM provides the ACI properties of ACID and since writing to a file provides the D ("durability"), it is tempting to think that STM plus a backup agent equals a database. This is *not* the case. A Clojure transaction promises only to send/sendoff an action to the agent; it does not actually *perform* the action under the ACI umbrella. So, for example, a transaction could complete, and then someone could unplug the power cord before the agent writes to the database. The moral is simple. If your problem calls for a real database, use a real database.

The Unified Update Model

As you have seen, refs, atoms, and agents all provide functions for updating their state by applying a function to their previous state. This unified model for handling shared state is one of the central concepts of Clojure. The unified model and various ancillary functions are summarized in the following table.

Update Mechanism	Ref Function	Atom Function	Agent Function
Function application	alter	swap!	send-off
Function (commutative)	commute	N/A	N/A
Function (nonblocking)	N/A	N/A	send
Simple setter	ref-set	reset!	N/A

The unified update model is by far the most important way to update refs, atoms, and agents. The ancillary functions, on the other hand, are optimizations and options that stem from the semantics peculiar to each API:

- The opportunity for the commute optimization arises when coordinating updates. Since only refs provide coordinated updates, commute makes sense only for refs.
- Updates to refs and atoms take place on the thread they are called on, so they provide no scheduling options. Agents update later, on a thread pool, making blocking/nonblocking a relevant scheduling option.

Clojure's final reference type, the var, is a different beast entirely. They do not participate in the unified update model and are instead used to manage thread-local, private state.

5.5 Managing Per-Thread State with Vars

When you call `def` or `defn`, you create a *dynamic var*, often called just a *var*. In all the examples so far in the book, you pass an initial value to `def`, which becomes the *root binding* for the var. For example, the following code creates a root binding for `foo` of 10:

```
(def ^:dynamic foo 10)
-> #'user/foo
```

The binding of `foo` is shared by all threads. You can check the value of `foo` on your own thread:

```
foo
-> 10
```

You can also verify the value of `foo` from another thread. Create a new thread, passing it a function that prints `foo`. Don't forget to start the thread:

```
user=> (.start (Thread. (fn [] (println foo))))
-> nil
| 10
```

In the previous example, the call to `start()` returns `nil`, and then the value of `foo` is printed from a new thread.

Most vars are content to keep their root bindings forever. However, you can create a *thread-local* binding for a var with the binding macro:

```
(binding [bindings] & body)
```

Bindings have *dynamic scope*. In other words, a binding is visible anywhere a thread's execution takes it, until the thread exits the scope where the binding began. A binding is not visible to any other threads.

Structurally, a binding looks a lot like a `let`. Create a thread-local binding for `foo` and check its value:

```
(binding [foo 42] foo)
-> 42
```

To see the difference between `binding` and `let`, create a simple function that prints the value of `foo`:

```
(defn print-foo [] (println foo))
-> #'user/print-foo
```

Now, try calling `print-foo` from both a `let` and a `binding`:

```
(let [foo "let foo"] (print-foo))
| 10

(binding [foo "bound foo"] (print-foo))
| bound foo
```

As you can see, the `let` has no effect outside its own form, so the first `print-foo` prints the root binding `10`. The binding, on the other hand, stays in effect down any chain of calls that begins in the binding form, so the second `print-foo` prints `bound foo`.

Acting at a Distance

Vars intended for dynamic binding are sometimes called *special* variables. It is good style to name them with leading and trailing asterisks. For example, Clojure uses dynamic binding for thread-wide options such as the standard I/O streams `*in*`, `*out*`, and `*err*`. Dynamic bindings enable *action at a distance*.

When you change a dynamic binding, you can change the behavior of distant functions without changing any function arguments.

One kind of action at a distance is temporarily augmenting the behavior of a function. In some languages this would be classified as aspect-oriented programming; in Clojure it is simply a side effect of dynamic binding. As an example, imagine that you have a function that performs an expensive calculation. To simulate this, write a function named `slow-double` that sleeps for a tenth of a second and then doubles its input.

```
(defn ^:dynamic slow-double [n]
  (Thread/sleep 100)
  (* n 2))
```

Next, write a function named `calls-slow-double` that calls `slow-double` for each item in `[1 2 1 2 1 2]`:

```
(defn calls-slow-double []
  (map slow-double [1 2 1 2 1 2]))
```

Time a call to `calls-slow-double`. With six internal calls to `slow-double`, it should take a little over six tenths of a second. Note that you will have to run through the result with `dorun`; otherwise, Clojure's `map` will outsmart you by immediately returning a lazy sequence.

```
(time (dorun (calls-slow-double)))
| "Elapsed time: 601.418 msecs"
-> nil
```

Reading the code, you can tell that `calls-slow-double` is slow because it does the same work over and over again. One times two is two, no matter how many times you ask.

Calculations such as `slow-double` are good candidates for *memoization*. When you memoize a function, it keeps a cache mapping past inputs to past outputs. If subsequent calls hit the cache, they will return almost immediately. Thus, you are trading space (the cache) for time (calculating the function again for the same inputs).

Clojure provides `memoize`, which takes a function and returns a memoization of that function:

```
(memoize function)
```

`slow-double` is a great candidate for memoization, but it isn't memoized yet, and clients like `calls-slow-double` already use the slow, unmemoized version. With dynamic binding, this is no problem. Simply create a binding to a memoized version of `slow-double`, and call `calls-slow-double` from within the binding.

```
(defn demo-memoize []
  (time
    (dorun
      (binding [slow-double (memoize slow-double)]
        (calls-slow-double))))))
```

With the memoized version of `slow-double`, `calls-slow-double` runs three times faster, completing in about two-tenths of a second:

```
(demo-memoize)
"Elapsed time: 203.115 msecs"
```

This example demonstrates the power and the danger of action at a distance. By dynamically rebinding a function such as `slow-double`, you change the behavior of *other* functions such as `calls-slow-double` without their knowledge or consent. With lexical binding forms such as `let`, it is easy to see the entire range of your changes. Dynamic binding is not so simple. It can change the behavior of other forms in other files, far from the point in your source where the binding occurs.

Used occasionally, dynamic binding has great power. But it should not become your primary mechanism for extension or reuse. Functions that use dynamic bindings are not pure functions and can quickly lose the benefits of Clojure's functional style.

Working with Java Callback APIs

Several Java APIs depend on callback event handlers. GUI frameworks such as Swing use event handlers to respond to user input. XML parsers such as SAX depend on the user implementing a callback handler interface.

These callback handlers are written with mutable objects in mind. Also, they tend to be single-threaded. In Clojure, the best way to meet such APIs halfway is to use dynamic bindings. This will involve mutable references that feel almost like variables, but because they are used in a single-threaded setting, they will not present any concurrency problems.

Clojure provides the `set!` special form for setting a thread-local dynamic binding:

```
(set! var-symbol new-value)
```

`set!` should be used rarely. In fact, the only place in the entire Clojure core that uses `set!` is the Clojure implementation of a SAX `ContentHandler`.

A `ContentHandler` receives callbacks as a parser encounters various bits of an XML stream. In nontrivial scenarios, the `ContentHandler` needs to keep track of

where it is in the XML stream: the current stack of open elements, current character data, and so on.

In Clojure-speak, you can think of a `ContentHandler`'s current position as a mutable pointer to a specific spot in an immutable XML stream. It is unnecessary to use references in a `ContentHandler`, since everything will happen on a single thread. Instead, Clojure's `ContentHandler` uses dynamic variables and `set!`. Here is the relevant detail:

```
; redacted from Clojure's xml.clj to focus on dynamic variable usage
(startElement
  [uri local-name q-name #^Attributes atts]
  ; details omitted
  (set! *stack* (conj *stack* *current*))
  (set! *current* e)
  (set! *state* :element))
nil)
(endElement
  [uri local-name q-name]
  ; details omitted
  (set! *current* (push-content (peek *stack*) *current*))
  (set! *stack* (pop *stack*))
  (set! *state* :between))
nil)
```

A SAX parser calls `startElement` when it encounters an XML start tag. The callback handler updates three thread-local variables. The `*stack*` is a stack of all the elements the current element is nested inside. The `*current*` is the current element, and the `*state*` keeps track of what kind of content is inside. (This is important primarily when inside character data, which is not shown here.)

`endElement` reverses the work of `startElement` by popping the `*stack*` and placing the top of the `*stack*` in `*current*`.

It is worth noting that this style of coding is the industry norm: objects are mutable, and programs are single-threadedly oblivious to the possibility of concurrency. Clojure permits this style as an explicit special case, and you should use it for interop purposes only.

The `ContentHandler`'s use of `set!` does not leak mutable data out into the rest of Clojure. Clojure uses the `ContentHandler` implementation to build an immutable Clojure structure.

You have now seen four different models for dealing with state. And since Clojure is built atop Java, you can also use Java's lock-based model. The models, and their use, are summarized in the following table.

Model	Usage	Functions
Refs and STM	Coordinated, synchronous updates	Pure
Atoms	Uncoordinated, synchronous updates	Pure
Agents	Uncoordinated, asynchronous updates	Any
Vars	Thread-local dynamic scopes	Any
Java locks	Coordinated, synchronous updates	Any

Let's put these models to work in designing a small but complete application.

5.6 A Clojure Snake

The Snake game features a player-controlled snake that moves around a game grid hunting for an apple. When your snake eats an apple, it grows longer by a segment, and a new apple appears. If your snake reaches a certain length, you win. But if your snake crosses over its own body, you lose.

Before you start building your own snake, take a minute to try the completed version. From the book's REPL, enter the following:

```
(use 'examples.snake)

(game)
-> [#<Ref clojure.lang.Ref@65694ee6>
    #<Ref clojure.lang.Ref@261ae209>
    #<Timer javax.swing.Timer@7f0df737>]
```

Select the Snake window, and use the arrow keys to control your snake.

Our design for the snake is going to take advantage of Clojure's functional nature and its support for explicit mutable state by dividing the game into three layers:

- The *functional model* will use pure functions to model as much of the game as possible.
- The *mutable model* will handle the mutable state of the game. The mutable model will use one or more of the reference models discussed in this chapter. Mutable state is much harder to test, so we will keep this part small.
- The *GUI* will use Swing to draw the game and to accept input from the user.

These layers will make the Snake easy to build, test, and maintain.

As you work through this example, add your code to the file `reader/snake.clj` in the sample code. When you open the file, you will see that it already imports/uses the Swing classes and Clojure libraries you will need:

```
src/reader/snake.clj
```

```
(ns reader.snake
  (:import (java.awt Color Dimension)
            (javax.swing JPanel JFrame Timer JOptionPane)
            (java.awt.event ActionListener KeyListener))
  (:use examples.import-static))
(import-static java.awt.event.KeyEvent VK_LEFT VK_RIGHT VK_UP VK_DOWN)
```

Now you are ready to build the functional model.

The Functional Model

First, create a set of constants to describe time, space, and motion:

```
(def width 75)
(def height 50)
(def point-size 10)
(def turn-millis 75)
(def win-length 5)
(def dirs { VK_LEFT  [-1  0]
            VK_RIGHT [ 1  0]
            VK_UP    [ 0 -1]
            VK_DOWN  [ 0  1]})
```

`width` and `height` set the size of the game board, and `point-size` is used to convert a game point into screen pixels. `turn-millis` is the heartbeat of the game, controlling how many milliseconds pass before each update of the game board. `win-length` is how many segments your snake needs before you win the game. (Five is a boringly small number suitable for testing.) The `dirs` maps symbolic constants for the four directions to their vector equivalents. Since Swing already defines the `VK_` constants for different directions, we will reuse them here rather than defining our own.

Next, create some basic math functions for the game:

```
(defn add-points [& pts]
  (vec (apply map + pts)))

(defn point-to-screen-rect [pt]
  (map #(* point-size %)
       [(pt 0) (pt 1) 1 1]))
```

The `add-points` function adds points together. You can use `add-points` to calculate the new position of a moving game object. For example, you can move an object at `[10, 10]` left by one:

Other Snake Implementations

There's more than one way to skin a snake. You may enjoy comparing the snake presented here with these other snakes:

- David Van Horn's Snake,^a written in Typed Scheme, has no mutable state.
- Jeremy Read wrote a Java Snake^b designed to be "just about as small as you can make it in Java and still be readable."
- Abhishek Reddy wrote a tiny (35-line) Snake^c in Clojure. The design goal was to be abnormally terse.
- Dale Vaillancourt's Worm Game.^d
- Mark Volkmann wrote a Clojure Snake^e designed for readability.

Each of the snake implementations has its own distinctive style. What would *your* style look like?

- <http://planet.plt-scheme.org/package-source/dvanhorn/snake.plt/1/0/main.ss>
- <http://www.plt1.com/1069/smaller-snake/>
- <http://www.plt1.com/1070/even-smaller-snake/>
- <http://www.ccs.neu.edu/home/cce/acl2/worm.html> includes some verifications using the theorem prover ACL2.
- <http://www.ocweb.com/mark/programming/ClojureSnake.html>

```
(add-points [10 10] [-1 0])
-> [9 10]
```

point-to-screen-rect simply converts a point in game space to a rectangle on the screen:

```
(point-to-screen-rect [5 10])
-> (50 100 10 10)
```

Next, let's write a function to create a new apple:

```
(defn create-apple []
  {:location [(rand-int width) (rand-int height)]
   :color (Color. 210 50 90)
   :type :apple})
```

Apples occupy a single point, the `:location`, which is guaranteed to be on the game board. Snakes are a little bit more complicated:

```
(defn create-snake []
  {:body (list [1 1])
   :dir [1 0]
   :type :snake
   :color (Color. 15 160 70)})
```

Because a snake can occupy multiple points on the board, it has a `:body`, which is a list of points. Also, snakes are always in motion in some direction expressed by `:dir`.

Next, create a function to move a snake. This should be a pure function, returning a new snake. Also, it should take a `grow` option, allowing the snake to grow after eating an apple.

```
(defn move [{:keys [body dir] :as snake} & grow]
  (assoc snake :body (cons (add-points (first body) dir)
                           (if grow body (butlast body)))))
```

`move` uses a fairly complex binding expression. The `{:keys [body dir]}` part makes the snake's body and `dir` available as their own bindings, and the `:as snake` part binds `snake` to the entire snake. The function then proceeds as follows:

1. `add-points` creates a new point, which is the head of the original snake offset by the snake's direction of motion.
2. `cons` adds the new point to the front of the snake. If the snake is growing, the entire original snake is kept. Otherwise, it keeps all the original snake except the last segment (`butlast`).
3. `assoc` returns a new snake, which is a copy of the old snake but with an updated `:body`.

Test `move` by moving and growing a snake:

```
(move (create-snake))
-> {:body ([2 1]), ; etc.

(move (create-snake) :grow)
-> {:body ([2 1] [1 1]), ; etc.
```

Write a `win?` function to test whether a snake has won the game:

```
(defn win? [{:body :body}]
  (>= (count body) win-length))
```

Test `win?` against different body sizes. Note that `win?` binds only the `:body`, so you don't need a "real" snake, just anything with a `body`:

```
(win? {:body [[1 1]]})
-> false

(win? {:body [[1 1] [1 2] [1 3] [1 4] [1 5]]})
-> true
```

A snake loses if its head ever comes back into contact with the rest of its body. Write a `head-overlaps-body?` function to test for this, and use it to define `lose?`:

```
(defn head-overlaps-body? [{[head & body] :body}]
  (contains? (set body) head))

(def lose? head-overlaps-body?)
```

Test `lose?` against overlapping and nonoverlapping snake bodies:

```
(lose? {:body [[1 1] [1 2] [1 3]]})
-> false

(lose? {:body [[1 1] [1 2] [1 1]]})
-> true
```

A snake eats an apple if its head occupies the apple's location. Define an `eats?` function to test this:

```
(defn eats? [{[snake-head] :body} {apple :location}]
  (= snake-head apple))
```

Notice how clean the body of the `eats?` function is. All the work is done in the bindings: `{[snake-head] :body}` binds `snake-head` to the first element of the snake's `:body`, and `{apple :location}` binds `apple` to the apple's `:location`. Test `eats?` from the REPL:

```
(eats? {:body [[1 1] [1 2]]} {:location [2 2]})
-> false

(eats? {:body [[2 2] [1 2]]} {:location [2 2]})
-> true
```

Finally, you need some way to turn the snake, updating its `:dir`:

```
(defn turn [snake newdir]
  (assoc snake :dir newdir))
```

`turn` returns a new snake, with an updated direction:

```
(turn (create-snake) [0 -1])
-> {:body ([1 1]), :dir [0 -1], ; etc.}
```

All of the code you have written so far is part of the functional model of the Snake game. It is easy to understand in part because it has no local variables and no mutable state. As you will see in the next section, the amount of *mutable* state in the game is quite small. (It is even possible to implement the Snake with *no* mutable state, but that is not the purpose of this demo.)

Building a Mutable Model with STM

The mutable state of the Snake game can change in only three ways:

- A game can be reset to its initial state.
- Every turn, the snake updates its position. If it eats an apple, a new apple is placed.
- A snake can turn.

We will implement each of these changes as functions that modify Clojure refs inside a transaction. That way, changes to the position of the snake and the apple will be synchronous and coordinated.

reset-game is trivial:

```
(defn reset-game [snake apple]
  (dosync (ref-set apple (create-apple))
          (ref-set snake (create-snake))))
  nil)
```

You can test reset-game by passing in some refs and then checking that they dereference to a snake and an apple:

```
(def test-snake (ref nil))
(def test-apple (ref nil))

(reset-game test-snake test-apple)
-> nil

@test-snake
-> {:body ([1 1]), :dir [1 0], ; etc.

@test-apple
-> {:location [52 8], ; etc.
```

update-direction is even simpler; it's just a trivial wrapper around the functional turn:

```
(defn update-direction [snake newdir]
  (when newdir (dosync (alter snake turn newdir))))
```

Try turning your test-snake to move in the “up” direction:

```
(update-direction test-snake [0 -1])
-> {:body ([1 1]), :dir [0 -1], ; etc.
```

The most complicated mutating function is update-positions. If the snake eats the apple, a new apple is created, and the snake grows. Otherwise, the snake simply moves:

```
(defn update-positions [snake apple]
  (dosync
    (if (eats? @snake @apple)
      (do (ref-set apple (create-apple))
          (alter snake move :grow))
      (alter snake move)))
  nil)
```

To test update-positions, reset the game:

```
(reset-game test-snake test-apple)
-> nil
```

Then, move the apple into harm's way, under the snake:

```
(dosync (alter test-apple assoc :location [1 1]))
-> {:location [1 1], ; etc.
```

Now, after you update-positions, you should have a bigger, two-segment snake:

```
(update-positions test-snake test-apple)
-> nil
```

```
(:body @test-snake)
-> ([2 1] [1 1])
```

And that is all the mutable state of the Snake world: three functions, about a dozen lines of code.

The Snake GUI

The Snake GUI consists of functions that paint screen objects, respond to user input, and set up the various Swing components. Since snakes and apples are drawn from simple points, the painting functions are simple. The fill-point function fills in a single point:

```
(defn fill-point [g pt color]
  (let [[x y width height] (point-to-screen-rect pt)]
    (.setColor g color)
    (.fillRect g x y width height)))
```

The paint multimethod knows how to paint snakes and apples:

```
Line 1 (defmulti paint (fn [g object & _] (:type object)))
2
3 (defmethod paint :apple [g {:keys [location color]}]
4   (fill-point g location color))
5
6 (defmethod paint :snake [g {:keys [body color]}]
7   (doseq [point body]
8     (fill-point g point color)))
```

paint takes two required arguments: `g` is a `java.awt.Graphics` instance, and `object` is the object to be painted. The `defmulti` includes an optional `rest` argument so that future implementations of `paint` have the option of taking more arguments. (See [Section 8.2, Defining Multimethods, on page 189](#) for an in-depth description of `defmulti`.) On line 3, the `:apple` method of `paint` binds the location and color of the apple and uses them to paint a single point on the screen. On line 6, the `:snake` method binds the snake's body and color and then uses `doseq` to paint each point in the body.

The meat of the UI is the `game-panel` function, which creates a Swing `JPanel` with handlers for painting the game, updating on each timer tick, and responding to user input:

```
Line 1 (defn game-panel [frame snake apple]
-   (proxy [JPanel ActionListener KeyListener] []
-     (paintComponent [g]
-       (proxy-super paintComponent g)
-       (paint g @snake)
-       (paint g @apple))
-     (actionPerformed [e]
-       (update-positions snake apple)
-       (when (lose? @snake)
-         (reset-game snake apple)
-         (JOptionPane/showMessageDialog frame "You lose!"))
-       (when (win? @snake)
-         (reset-game snake apple)
-         (JOptionPane/showMessageDialog frame "You win!"))
-       (.repaint this))
-     (keyPressed [e]
-       (update-direction snake (dirs (.getKeyCode e))))
-     (getPreferredSize []
-       (Dimension. (* (inc width) point-size)
-                    (* (inc height) point-size)))
-     (keyReleased [e])
-     (keyTyped [e])))
```

`game-panel` is long but simple. It uses `proxy` to create a panel with a set of Swing callback methods.

- Swing calls `paintComponent` (line 3) to draw the panel. `paintComponent` calls `proxy-super` to invoke the normal `JPanel` behavior, and then it paints the snake and the apple.
- Swing will call `actionPerformed` (line 7) on every timer tick. `actionPerformed` updates the positions of the snake and the apple. If the game is over, it displays a dialog and resets the game. Finally, it triggers a repaint with `(.repaint this)`.

- Swing calls `keyPressed` (line 16) in response to keyboard input. `keyPressed` calls `update-direction` to change the snake's direction. (If the keyboard input is not an arrow key, the `dirs` function returns `nil` and `update-direction` does nothing.)
- The game panel ignores `keyReleased` and `keyTyped`.

The game function creates a new game:

```
Line 1 (defn game []
-   (let [snake (ref (create-snake))
-         apple (ref (create-apple))
-         frame (JFrame. "Snake")
5       panel (game-panel frame snake apple)
-         timer (Timer. turn-millis panel)]
-     (doto panel
-       (.setFocusable true)
-       (.addKeyListener panel))
10    (doto frame
-      (.add panel)
-      (.pack)
-      (.setVisible true))
-      (.start timer)
15    [snake, apple, timer]))
```

On line 2, `game` creates all the necessary game objects: the mutable model objects `snake` and `apple` and the UI components `frame`, `panel`, and `timer`. Lines 7 and 10 perform boilerplate initialization of the panel and frame. Line 14 starts the game by kicking off the timer.

Line 15 returns a vector with the snake, apple, and time. This is for convenience when testing at the REPL: you can use these objects to move the snake and apple or to start and stop the game.

Go ahead and play the game again; you have earned it. To start the game, use the snake library at the REPL, and run `game`. If you have entered the code yourself, you can use the library name you picked (examples.reader in the instructions); otherwise, you can use the completed sample at `examples.snake`:

```
(use 'examples.snake)

(game)
-> [#<Ref clojure.lang.Ref@6ea27cbe>
   #<Ref clojure.lang.Ref@6dabd6b0>
   #<Timer javax.swing.Timer@32f60451>]
```

The game window may appear behind your REPL window. If this happens, use your local operating-system `fu` to locate the game window.

There are many possible improvements to the Snake game. If the snake reaches the edge of the screen, perhaps it should turn to avoid disappearing from view. Or (tough love) maybe you just lose the game! Make the Snake game your own by improving it to suit your personal style.

Snakes Without Refs

We chose to implement the Snake game's mutable model using refs so that we could coordinate the updates to the snake and the apple. Other approaches are also valid. For example, you could combine the snake and apple state into a single game object. With only one object, coordination is no longer required, and you can use an atom instead.

The file `examples/atom-snake.clj` demonstrates this approach. Functions like `update-positions` become part of the functional model and return a new game object with updated state:

```
src/examples/atom_snake.clj
(defn update-positions [{snake :snake, apple :apple, :as game}]
  (if (eats? snake apple)
      (merge game {:apple (create-apple) :snake (move snake :grow)})
      (merge game {:snake (move snake)})))
```

Notice how destructuring makes it easy to get at the internals of the game: both `snake` and `apple` are bound by the argument list.

The actual mutable updates are now all atom swap!s. We found these to be simple enough to leave them in the UI function `game-panel`, as this excerpt shows:

```
(actionPerformed [e]
  (swap! game update-positions)
  (when (lose? (@game :snake))
    (swap! game reset-game)
    (JOptionPane/showMessageDialog frame "You lose!")))
```

There are other possibilities as well. Chris Houser's fork of the book's sample code² demonstrates using an agent that `Thread/sleeps` instead of a `Swing timer`, as well as using a new agent per game turn to update the game's state.

5.7 Wrapping Up

Clojure's reference model is the most innovative part of the language. The combination of software transactional memory, agents, atoms, and dynamic

2. <http://github.com/Chouser/programming-clojure>

binding that you have seen in this chapter gives Clojure powerful abstractions for all sorts of stateful systems. It also makes Clojure one of the few languages suited to the coming generation of multicore computer hardware.

Next, we will look at one of Clojure's newer features. Some call it a solution to the "expression problem."³ We call it a protocol.

3. http://en.wikipedia.org/wiki/Expression_problem